

AWS Security Monitoring & Alerting: Secrets Manager Access Detection + Automated Kill-Switch

Technical Documentation &
Installation Guide

Iniobong Johnson

February 2026

Project Objective: The objective of this project is to implement a security monitoring and alerting system that detects access to sensitive AWS resources (in this case, our AWS Secrets Manager) and provides real-time notifications to the subscribed SNS protocol.

Phase 1: Infrastructure & Logging

Task 1.1: Create AWS CloudTrail (Multi-Region)

- Navigate to CloudTrail using the search bar on AWS Console >> click Trail >> Create trail.
- Give Trail a name (SM-SecurityMonitoring-Trail)
- Create an S3 bucket >> give it a name >> Uncheck SSE-KMS encryption (to avoid charges).
- Enable Log file integrity validation
- Enable CloudWatch Logs >> Give Log group a name
- IAM Role (use the default) >> Give the IAM Role a name >> Next
- Event type (use the default - Management events) >> API activity (Check the Read and Write and also check the listed excluded option (this prevents high-volume KMS logs and focuses logs on main security events)) >> Next
- Review and click Create Trail

Task 1.2: Create a Secret in AWS Secrets Manager

- Navigate to AWS Secrets Manager using the search bar
- On the dashboard click on Store a new secret
- Secret type (select other type of secret)
- Enter a Key/value ((add rows as needed, e.g., key: api_key, value: your-value-here) or switch to Plaintext tab if preferred) >> Leave the default Encryption key >> Next
- Give it a Secret name >> Add Description >> Next
- Leave the Configure automatic rotation as disabled >> Next
- Review >> Store

Task 1.3: Enable Server Access Logging

- Navigate to S3 bucket using the search bar.
- Create a new S3 bucket (it avoids recursive logging loops and keeps your CloudTrail log bucket clean)
- Select the CloudTrail Log bucket >> click Properties tab
- Scroll to Server Access Logging >> click Edit >> Enable Server access logging >> and select the new bucket as the target.
- Save changes.

Verified CloudTrail start collecting Logs

- Wait a few minutes for logs to generate

- Retrieve the secret from the Secrets Manager console (go to Secrets >> select your secret >> click Retrieve secret value).
- retrieving the secret from Secret manager page
- Visit Cloudtrail >> Event history >> filter by Event source: secretsmanager.amazonaws.com or Event name: GetSecretValue >> confirm an entry for accessing the secret..
- Verify all logs are inside the Log group. In CloudWatch Logs >> go to your log group >> confirm CloudTrail events (including the secret access) are appearing in log streams.

Phase 2: Detection Logic - Flow 1

Task 2.1: Create Metric Filter

- Navigate to CloudWatch >> Expand Logs >> Log Management >> select the created Log group
- Click on Metric Filter >> Create metric filter
- Enter filter pattern name >> click Next
- Give it a filter name >> Metric namespace >> Metric name >> Metric Value >> click Next
- Review and click on Create metric filter

Task 2.2: Create CloudWatch Alarm

- After creating the Metric filter(from Task 2.1), stay in the log group >>In the Metric filter tab, click on the checkbox next to the new metric filter >> click on Create alarm
- Under Specify metric and condition, set the Statistic value to sum >> Period set to 1 minute
- Set the Threshold type to Static >> Define the alarm condition (Greater/Equal 1) >> click on Next

Task 2.3: Set up SNS Notification

- Create new topic >> Give it a Topic Name >> Enter the Endpoint (Email) >> click Next
- Give the alarm a name >> click Next
- Review and click Create alarm

Notes: After creation, check your email for the SNS confirmation link and click it to activate subscriptions.

Phase 3: Detection Logic - Flow 2 [Amazon EventBridge]

Task 3.1: Create Amazon EventBridge

- Open AWS Console >> Search Amazon EventBridge >> Rules >> Create rule
- Give the Rule a name >>add Rule description >> leave Event Bus as default >> click on Next

- Select Event (AWS events or EventBridge partner events) >> Creation method >> Event source >> AWS service >> Event type >> Event type specification 1 (select Specific operation) and enter the Specific Operation

Enter the JSON script under the Event pattern

```
{
  "source": ["aws.secretsmanager"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["secretsmanager.amazonaws.com"],
    "eventName": ["GetSecretValue"]
  }
}
```

- On the Configure page, give the rule a Name >> Description >> Event bus (leave as default)

Task 3.2: Creating a Separate SNS topic

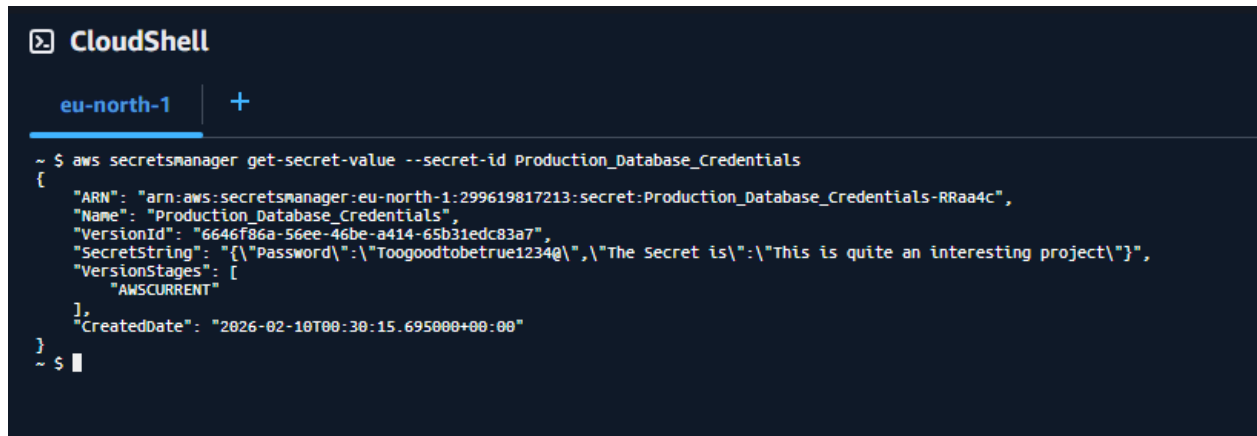
- On the SNS dashboard, click Topic >> Give the Topic a Name >> Create topic
- Create Subscription >> Select Protocol >> Enter the Endpoint >> Create subscription
- Set the New SNS as target to EventBridge
 - Go to Amazon EventBridge >> click on Rule >> Select your rule (The created Rule from Task 3.1)
 - Click Edit >> Scroll to Targets section >> Click add Target >> Target type (select SNS Topic: Choose the new topic)
 - Click Update rule or Save

Notes: After creation, check your email for the SNS confirmation link and click it to activate subscriptions.

Task 3.3: Breach Observation using AWS CLI

- Open your terminal or AWS CloudShell with appropriate IAM permissions
- Run the command to retrieve the secret value (simulates a breach/access)

```
aws secretsmanager get-secret-value --secret-id Production_Database_Credentials
```



```
CloudShell
eu-north-1 +
~ $ aws secretsmanager get-secret-value --secret-id Production_Database_Credentials
{
  "ARN": "arn:aws:secretsmanager:eu-north-1:299619817213:secret:Production_Database_Credentials-RRaa4c",
  "Name": "Production_Database_Credentials",
  "VersionId": "6646f86a-56ee-46be-a414-65b31edc83a7",
  "SecretString": "{\"Password\":\"Toogoodtobetruer1234@\", \"The Secret is\":\"This is quite an interesting project\"}",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2026-02-10T00:30:15.695000+00:00"
}
~ $
```

Challenges Encountered:

- **CloudTrail logs the event successfully:** The GetSecretValue event appeared correctly in CloudTrail Event history after running the retrieval command.
- **EventBridge rule did not trigger notification initially:** By default, when creating an EventBridge rule in the console for "AWS API Call via CloudTrail" events, the rule state is set to **ENABLED**.
 - **ENABLED** only matches write management events (e.g., PutSecretValue, CreateSecret).
 - **GetSecretValue** is a read-only management event.
- Since November 2023, AWS supports read-only CloudTrail events in EventBridge, but the rule must be explicitly set to **ENABLED_WITH_ALL_CLOUDTRAIL_MANAGEMENT_EVENTS** to include them.

Resolution: Update rule state via AWS CLI

Used the following command to force-enable the rule for all management events (including read-only):

```
aws events put-rule \
  --name SensitiveDataAccessRule \
  --event-pattern '{"source":["aws.secretsmanager"],"detail-type":["AWS API Call via CloudTrail"],"detail":{"eventSource":["secretsmanager.amazonaws.com"],"eventName":["GetSecretValue"],"requestParameters":{"secretId":["Production_Database_Credentials"]}}}' \
  --state ENABLED_WITH_ALL_CLOUDTRAIL_MANAGEMENT_EVENTS \
  --description "Triggers on GetSecretValue for Production_Database_Credentials honeypot" \
  --region eu-north-1
```

- After running this, the rule state changed to **ENABLED_WITH_ALL_CLOUDTRAIL_MANAGEMENT_EVENTS**.
- Re-tested secret retrieval >> EventBridge now matches the pattern >> SNS notification sent successfully.

Phase 4: The “Active Defense” Kill-Switch

Task 4.1: The Target, Create the Response Mechanism

Choose one: Lambda Function or SSM

Create the Lambda Function in Console

- Go to AWS Console >> Lambda >> Functions >> Create function
- Author from scratch >> Function name: >> Runtime: Python 3.12 (or latest) >>
- Architecture: x86_64 >> Click Create function

Add Code to the Function

In the Code tab, the default was replaced with the Python code below (It parses the EventBridge event to get the IAM username, then detaches all attached managedpolicies)

Python Code Used

```
import json
import boto3

def lambda_handler(event, context):
    iam = boto3.client('iam')

    # Extract username from EventBridge/CloudTrail event
    user_arn = event['detail']['userIdentity']['arn']
    username = user_arn.split('/')[-1] # e.g., 'victim-user'

    # List and detach all attached policies
    attached_policies =
iam.list_attached_user_policies(UserName=username)['AttachedPolicies']
    for policy in attached_policies:
        iam.detach_user_policy(UserName=username, PolicyArn=policy['PolicyArn'])

    print(f"Revoked permissions for user: {username}")
    return {
        'statusCode': 200,
        'body': json.dumps('Permissions revoked successfully')
    }
```

```
}
```

Click Deploy

Add IAM Permissions to Lambda's Role

- In Lambda >> Configuration tab >> Permissions >> click the Execution role name >> View in IAM (by clicking the clickable link next to the role name)
- Attach policies by clicking on Add permission >> Choose Attach policies >> Search for the policy (IAMFullAccess) >> Click on Add permission

Task 4.2: The Target, Connect to EventBridge Rule

- Add the Lambda as a second target on the phase 3 EventBridge rule (beside the SNS Topic)
- Edit the Rule in the Console
- Go to EventBridge >> Rules >> Select the created rule >> Edit
- In the Target section, add a new target (since we are using Lambda), select AWS Service >> Lambda function >> your function (RevokeAccessLambda)
- Grant permission (EventBridge will prompt you to add invoke permissions) >> Save rules

Challenge Encountered

- Editing the rule in the console to add the Lambda target was blocked.
- Cause: The rule state was set to **ENABLED_WITH_ALL_CLOUDTRAIL_MANAGEMENT_EVENTS** to support read-only events like GetSecretValue. Rules in this state have limited console edit support for targets

Resolution: Add Target and Permission via AWS CLI

1. Add the the Lambda as target

```
aws events put-targets \
  --rule SensitiveDataAccessRule \
  --targets
Id=lambda-target,Arn=arn:aws:lambda:eu-north-1:299619817213:function:Lambdafunctionnam
e \
  --region eu-north-1
```

It will return the below

```
{
  "FailedEntryCount": 0,
```

```
"FailedEntries": []  
}
```

FailedEntryCount: 0 means everything was added successfully.

2. Grant EventBridge permission to invoke Lambda

```
aws lambda add-permission \  
  --function-name Lambdafunctionname \  
  --statement-id EventBridgeInvoke \  
  --action lambda:InvokeFunction \  
  --principal events.amazonaws.com \  
  --source-arn arn:aws:events:eu-north-1:299619817213:rule/SensitiveDataAccessRule \  
  --region eu-north-1
```

3. Verify the target was added

```
aws events list-targets-by-rule --rule SensitiveDataAccessRule --region eu-north-1
```

Task 4.3: The Trap Test

- Create IAM User (Victim):
- IAM Console >> Users >> Add user
- Give the IAM a name >> Access (AWS Management Console (password)) >> Permission (SecretsManagerReadWrite)
- Click Next >> Review >> Create User
- On the Created user page click on Create access key >> choose a Use case (Command Line Interface (CLI)) >> Confirm the recommendation warning >> Next
- Add a Description tag value >> Create access key
- Immediately note/download the keys (Access and Secret access keys)

NOTE: The access key lets you impersonate the compromised/victim user from the command line to safely trigger the detection + auto-remediation (kill-switch). Without it, you can't run the CLI test as the VictimUser.

Configure AWS CLI Profile for Victim

```
aws configure --profile victim
```

You will be prompted to enter the following:

- AWS Access Key ID: [your access key ID]
- AWS Secret Access Key: [your secret key]

- Default region name: eu-north-1
- Default output format: json

NOTE: It lets you run AWS CLI commands as the VictimUser without affecting your normal (admin) credentials.

Then test the Kill-Switch using

```
aws secretsmanager get-secret-value --secret-id Production_Database_Credentials --profile victim --region eu-north-1
```

```
aws s3 ls --profile victim --region eu-north-1
```

Phase 5: Final Forensic Report

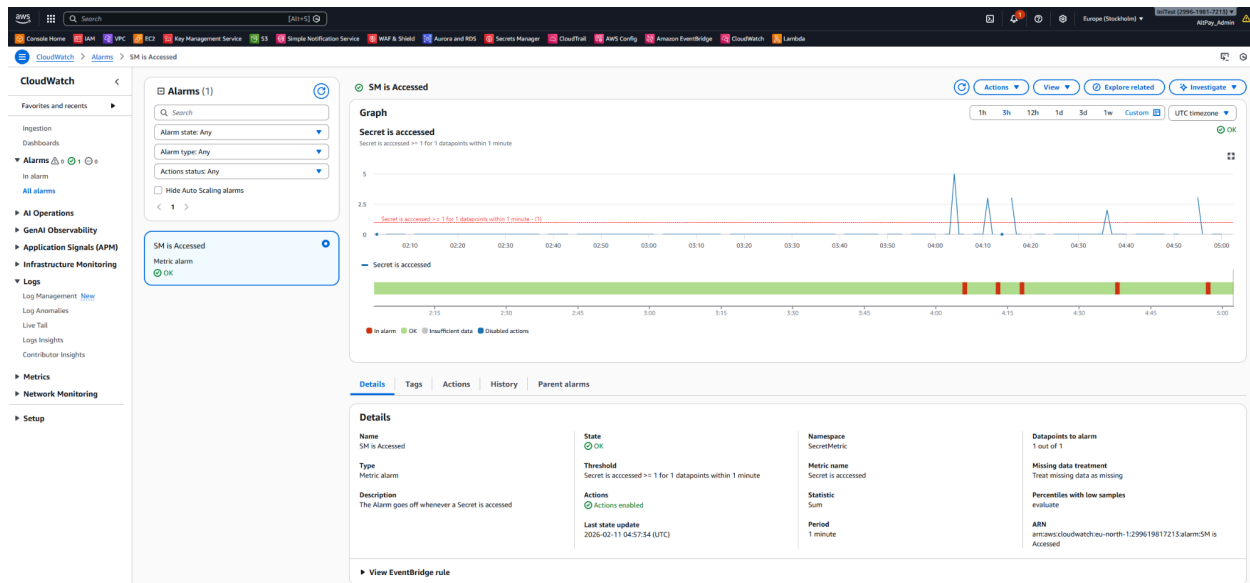
Executive Summary

Goal: Build a security monitoring system in AWS that detects sensitive access and alerts in real time.

Outcome: The system successfully detected secret access, triggered alert via CloudWatch, EventBridge and automatically revoked the compromised IAM user's permissions

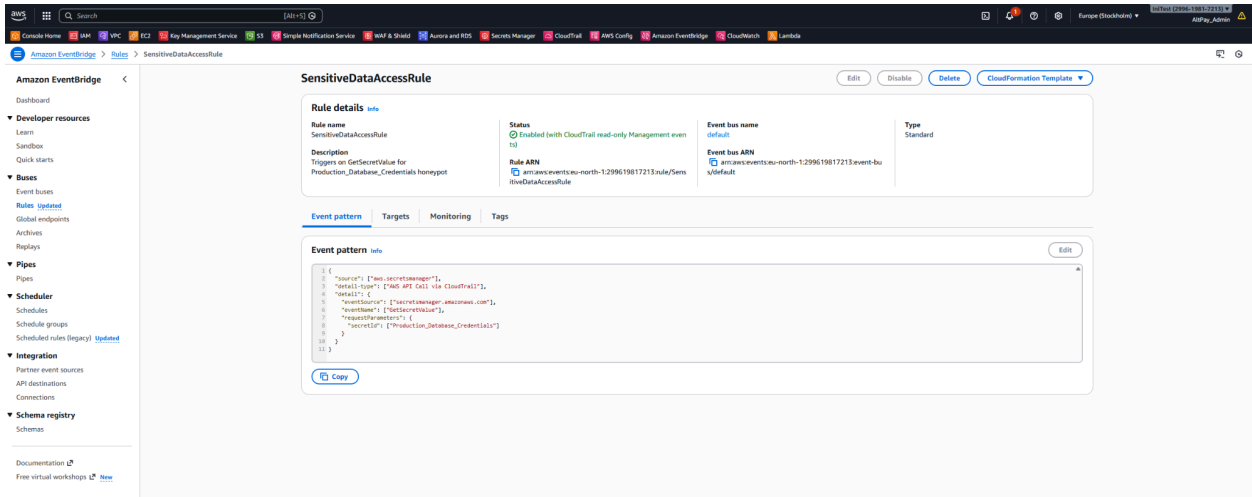
Lab Architecture

CloudWatch Alarm Configuration

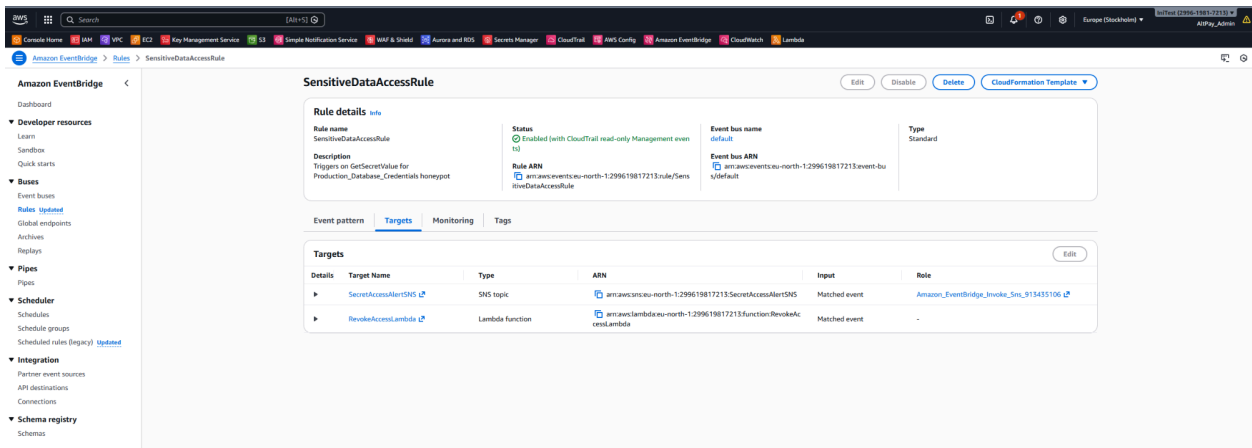


EventBridge Rule

Event Pattern



Targets



Evidence Collected:

CloudTrail Log Snippet:

Entire Log

Event name	Event time	User name	Event source	Resource type	Resource name
GetSecretValue	February 11, 2026, 05:14:30 (UTC+...)	VictimUser	secretmanager.amazonaws.com	-	-
GetSecretValue	February 11, 2026, 05:09:36 (UTC+...)	VictimUser	secretmanager.amazonaws.com	AWS-SecretsManager...	Production_Database_Credentials
GetSecretValue	February 11, 2026, 05:05:05 (UTC+...)	VictimUser	secretmanager.amazonaws.com	AWS-SecretsManager...	Production_Database_Credentials
GetSecretValue	February 11, 2026, 01:06:19 (UTC+...)	AitPay_Admin	secretmanager.amazonaws.com	AWS-SecretsManager...	Production_Database_Credentials
GetSecretValue	February 10, 2026, 22:22:14 (UTC+...)	AitPay_Admin	secretmanager.amazonaws.com	AWS-SecretsManager...	Production_Database_Credentials
GetSecretValue	February 10, 2026, 04:51:15 (UTC+...)	AitPay_Admin	secretmanager.amazonaws.com	AWS-SecretsManager...	Production_Database_Credentials
GetSecretValue	February 10, 2026, 04:28:38 (UTC+...)	AitPay_Admin	secretmanager.amazonaws.com	AWS-SecretsManager...	Production_Database_Credentials
GetSecretValue	February 10, 2026, 01:53:57 (UTC+...)	AitPay_Admin	secretmanager.amazonaws.com	AWS-SecretsManager...	arn:aws:secretmanager:eu-north-1:299619817213:secret:Production_Database_Credentials-RR2a4c

Attacker JSON Log Snippet

```

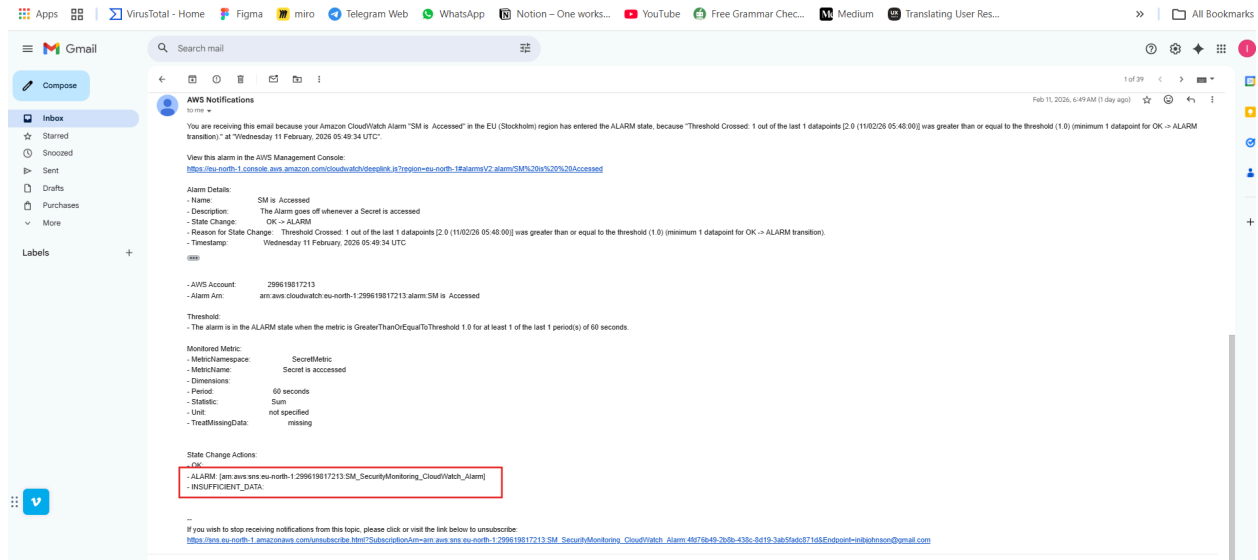
1 {
2   "eventVersion": "1.11",
3   "userIdentity": {
4     "type": "IAMUser",
5     "principalId": "AIDAILQV35W4PQ3P4H6",
6     "arn": "arn:aws:iam::299619817213:user/VictimUser",
7     "accountId": "299619817213",
8     "accessKeyId": "AKIAIOLQV35W4PQ3P4H6",
9     "userName": "VictimUser"
10  },
11  "eventTime": "2026-02-11T04:09:36Z",
12  "eventSource": "secretmanager.amazonaws.com",
13  "eventName": "GetSecretValue",
14  "awsRegion": "eu-north-1",
15  "sourceIPAddress": "75.90.8.207",
16  "userAgent": "aws-cdk-v2.100.0-rc.2026-02-11 ua/2.1 os/linux#6.1.159-181.297.amzn2023.x86_64 md/arch#x86_64 lang/python#3.11.11 md/pyip#IPython exec-emu/CloudShell w/n,t,z,b cfg/retry-mode#standard md/installer#aws md/distribution#2023 md/prompt#ff md/command#secretmanager.getSecretValue",
17  "requestParameters": {
18    "secretId": "Production_Database_Credentials"
19  },
20  "responseElements": null,
21  "requestID": "7231f854-26cc-43ca-b3ae-98b0b187905",
22  "eventID": "03616032-b0ca-ae25-23c380a6-339",
23  "readOnly": true,
24  "eventType": "AwsApiCall",
25  "managementEvent": true,
26  "recipientAccountId": "299619817213",
27  "eventCategory": "Management",
28  "details": {
29    "serviceName": "SecretsManager",
30    "apiVersion": "2018-08-01",
31    "clientId": "arn:aws:iam::299619817213:user/VictimUser",
32    "clientProviderHostHeader": "secretmanager.eu-north-1.amazonaws.com",
33    "keyExchange": "x25519"
34  }
35 }

```

Technical Evidence Screenshots:

The Detection:

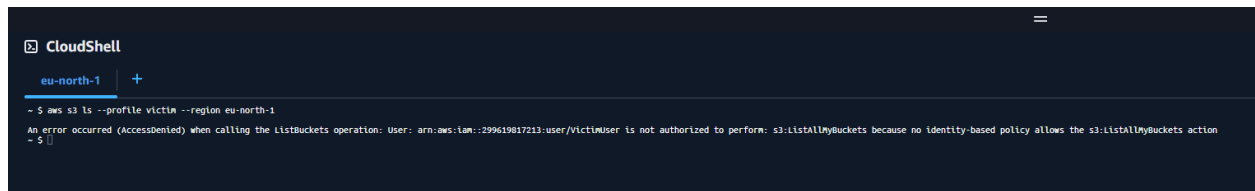
Flow 1 Email Notification



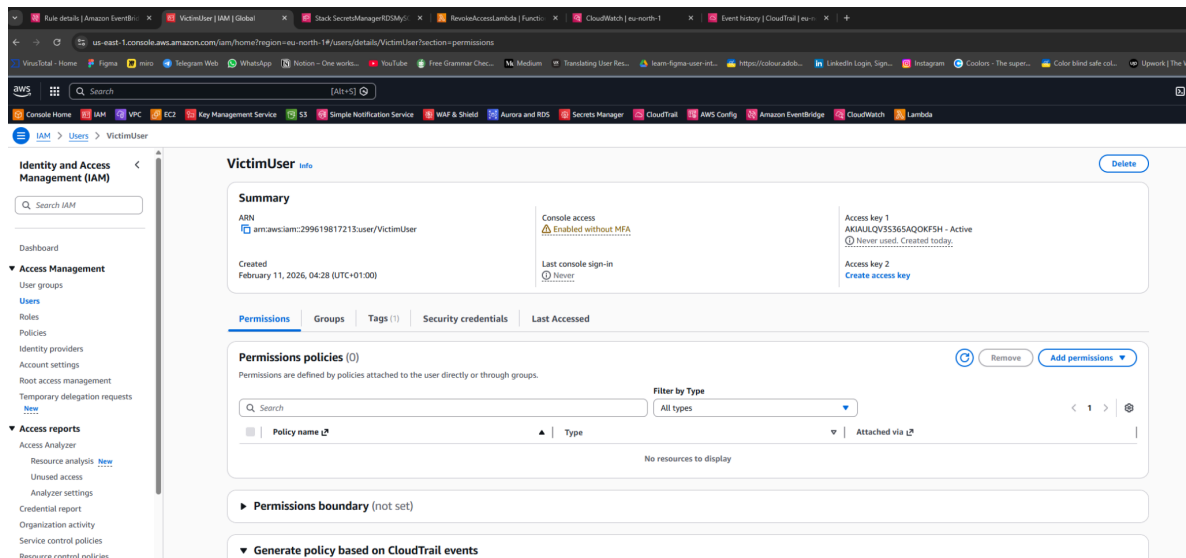
The “Kill-Switch” Proof:

The kill-switch successfully revoked permissions: The victim user received AccessDenied on subsequent actions due to no remaining identity-based policies allowing any actions.

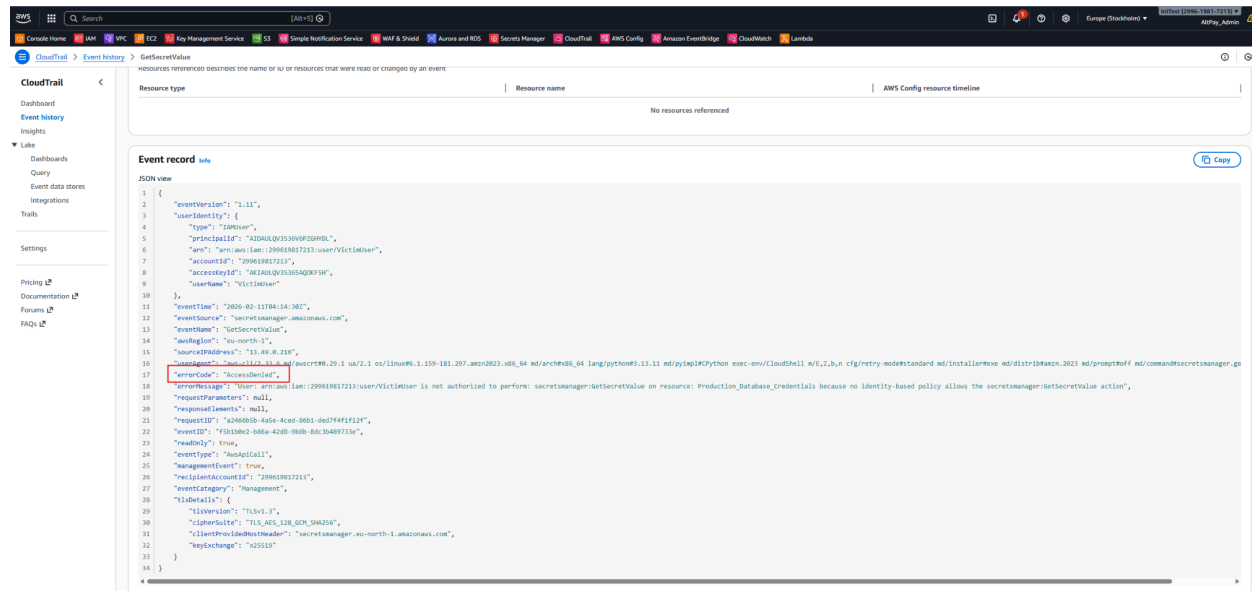
Proof of Revocation CloudShell:



IAM Permission Tab:



The Log snippet of the CloudTrail JSON showing the errorCode: AccessDenied for the user's *second* attempt to use the CLI



Security Analysis

- **Speed:** EventBridge (Flow 2) delivered notifications significantly faster than CloudWatch alarms. The time difference was substantial. This demonstrates that EventBridge is better suited for scenarios where immediate detection and response are critical.
- **Context:**
 - EventBridge/Lambda provides more details informations like user, IP, eventTime in logs.
 - After the alert is triggered (via CloudWatch Alarm in Flow 1 or EventBridge rule in Flow 2), the most effective and commonly used AWS service to automatically block/revoke access for the user would be AWS Lambda, invoked directly as a target from the EventBridge rule.
- **Compliance:** This lab satisfies NIST Continuous Monitoring requirements by using CloudTrail and EventBridge to provide ongoing visibility into sensitive data access events. EventBridge rules continuously monitor for Secrets Manager **GetSecretValue** calls, triggering alerts via SNS and enabling automated responses through Lambda. This aligns with NIST by ensuring continuous detection, real-time awareness, and enforcement of security controls.

Lessons Learned

When working on a project that requires security and monitoring, it is important to test all processes to ensure data accuracy. Errors may occur, but researching and calmly troubleshooting helps in understanding and resolving issues effectively.

For a bank, I would prefer to use **both Metric Filters and EventBridge** together. Each has unique strengths that complement one another:

- **Metric Filters** convert raw log data into measurable metrics, allowing the bank to define patterns and set alarms for suspicious activity. This supports long-term monitoring and trend analysis.
- **EventBridge** provides real-time detection and notification, enabling immediate awareness and rapid response to potential threats.

By combining both, a bank can achieve continuous monitoring that not only detects anomalies quickly but also tracks them over time, ensuring a layered defense against threats.